

System Architecture Document: AI Email Processing Agent

1. System Overview

This document outlines the architecture for an automated, AI-driven email processing agent. The system is designed to asynchronously fetch unread emails, sanitize sensitive information, classify user intent, draft context-aware responses using AI, and persist the data while providing a user interface for monitoring and analytics. The architecture is highly decoupled, utilizing message queues, asynchronous workers, and dedicated observability pipelines to ensure scalability and reliability.

2. Module Explanations

API Gateway (Green Boundary)

Function: Acts as the entry point for both automated scheduled tasks and client requests.

- **Cronjob Orchestrator:** An automated scheduler that routinely queries external email services (via Microsoft Graph API) to find unread emails. It acts as the initiator for the automated email pipeline.
- **Security & Traffic Management:** Handles incoming API requests (e.g., from the web dashboard). It routes requests through several middleware layers.
- **PII & Decryption Layer:** Ensures incoming payloads are secure and readable.
- **Security Middleware (Auth):** Validates user credentials and permissions.
- **Traffic Control Layer:** Prevents system abuse and ensures high availability by capping request rates through rate limiting and throttling.

Message Broker (Red Boundary)

- **Upstash Redis Queue:** Serves as the intermediary between the API Gateway and the Asynchronous Engine. When the orchestrator finds new emails, it packages the Mail IDs as tasks and pushes them into this Redis queue. This decouples email discovery from the heavy AI processing.

Asynchronous Engine & AI Processing Pipeline (Purple & Blue Boundaries)

Function: This is the core compute engine of the system, responsible for the heavy lifting of natural language processing and drafting.

- **Auto-Scaled Celery Workers:** Background worker processes that poll the Upstash Redis Queue for new jobs. They scale dynamically based on the queue size and trigger the AI Processing Pipeline.
- **Step 1 - Ingestion (Graph Connector):** Receives the Mail ID task and reaches back out to the Microsoft Graph API to fetch the raw text of the email body.
- **Step 2 - Parse & Sanitization (Presidio Masker):** Processes the raw text to detect and mask Personally Identifiable Information (PII) to ensure data privacy before AI processing. Outputs a masked body.
- **Step 3 - Intent Classifier:** Analyzes the masked text to determine the core purpose of the user's email, outputting intent & metrics.
- **Step 4 - Customer Verifier:** Cross-references the sender/intent with the PostgreSQL DB (verifying active plans/status) to retrieve specific customer status info.
- **Step 5 - Query Resolver:** Gathers all previous contexts to determine the factual answer or resolution required for the email.
- **Step 6 - Email Drafter:** Synthesizes the resolved query, customer context, and pre-defined mail templates (fetched from External Systems) to generate a customized draft response.
- **Step 7 - DB Connector (Persistence):** The final step in the pipeline. It saves the processed draft and associated metadata to the Data Infrastructure. It also signals the External System to mark the original email as "Read!".

Data Infrastructure (Yellow Boundary)

Function: Handles all persistence, analytics, and record-keeping.

- **PostgreSQL DB:** The primary relational database. It stores user data, usage metrics, and processed email rows.
- **Audit Logs:** A dedicated storage mechanism for compliance and troubleshooting, tracking system actions.
- **Data Analytics:** An analytical processing layer that syncs with the database to generate business intelligence and usage insights.

External Systems (Grey Boundary)

Function: Third-party APIs and external storage.

- **Microsoft Graph API:** Used to interface with the enterprise email server (Exchange/Outlook) to read emails and update their read/unread status.
- **Templates:** A repository of standard email templates utilized by the Email Drafter to maintain brand voice and formatting.

Client Tier (Blue Boundary, Bottom)

- **React Dashboard:** The front-end user interface. It connects to the PostgreSQL database to fetch user data and metrics, and sends API requests back through the API Gateway for user-driven actions.

Observability Stack (Orange Boundary)

- **OpenTelemetry:** Collects standardized metrics, logs, and traces from across the system (specifically API Telemetry from the Gateway and Worker Telemetry from the Celery workers).
- **Grafana (Visualize):** A visualization dashboard used by system administrators and engineers to monitor system health, queue lengths, and error rates in real-time.

3. End-to-End Workflow

The automated lifecycle of an email through this system follows these precise steps:

Phase 1: Discovery & Queuing

1. **Request Mails:** The Cronjob Orchestrator (API Gateway) routinely pings the Microsoft Graph API requesting the IDs of any unread emails.
2. **Return IDs:** The Microsoft Graph API returns a list of unread Mail IDs.
3. **Push to Queue:** The Orchestrator packages these IDs as distinct tasks and pushes them into the Upstash Redis Queue.

Phase 2: Processing & Drafting (The AI Pipeline)

1. **Poll Job:** The Auto-Scaled Celery Workers continuously poll the queue, pick up the pending Mail ID tasks, and trigger the AI flow.
2. **Fetch Content:** The Graph Connector takes the ID and fetches the actual raw text body from the Microsoft Graph API.
3. **Sanitize:** The raw text is passed through the Presidio Masker to strip out sensitive PII.
4. **Classify & Verify:** The masked text goes to the Intent Classifier to determine what the user wants. Simultaneously, the Customer Verifier queries the PostgreSQL DB to check the customer's subscription or account status.

5. **Resolve & Draft:** The Query Resolver establishes the facts needed for a reply. The Email Drafter then pulls a relevant template from the external repository and uses AI to write a contextual response based on the resolved query.

Phase 3: Persistence & Cleanup

1. **Save State:** The DB Connector saves the finalized draft and process state into the PostgreSQL DB and writes an entry to the Audit logs.
2. **Mark as Read:** The pipeline sends a final command to the Microsoft Graph API to mark the processed email as "Read".

Phase 4: Monitoring (Continuous)

1. **Telemetry Stream:** Throughout the entire process, telemetry data (worker health, API latency, success rates) is continuously streamed to OpenTelemetry and visualized on Grafana for operational oversight.